

Texto do aluno e lista de atividades

Encontro 01 — Fundamentos e objetivos da criptografia

Duas aulas semanais (100 minutos)

Glossário de siglas, termos técnicos e expressões em inglês

Consulte esta seção antes da leitura. As definições apresentam o sentido adotado neste material.

- **AEAD** — *Authenticated Encryption with Associated Data* (cifração autenticada com dados associados): protege sigilo e integridade numa única construção.
- **algoritmo** — Sequência finita e bem definida de operações para resolver um problema ou realizar uma transformação.
- **assert** — Comando de asserção do Python: interrompe a execução quando uma condição esperada é falsa e, neste material, funciona como teste automatizado.
- **ataque ativo** — Ataque que altera, injeta, remove, reorganiza ou repete mensagens.
- **ataque passivo** — Ataque que observa dados ou metadados sem modificar diretamente a comunicação.
- **ativo** — Informação, serviço, equipamento ou processo que possui valor e precisa de proteção.
- **autenticação** — Processo de verificar uma identidade alegada ou a origem de uma mensagem.
- **autenticidade** — Propriedade que fornece evidência sobre a origem de dados, mensagens ou ações.
- **autorização** — Decisão sobre quais ações uma identidade autenticada tem permissão para executar.
- **backup** — Cópia de segurança usada para recuperar dados após perda, corrupção ou indisponibilidade.
- **Base64** — Codificação que representa bytes por caracteres imprimíveis; é reversível sem chave e não oferece criptografia.
- **biblioteca de software** — Conjunto reutilizável de funções e classes que um programa chama por uma interface definida.
- **bit** — Unidade binária de informação que pode assumir os valores 0 ou 1.
- **byte** — Grupo de oito bits, frequentemente usado como unidade de dados.
- **chave** — Valor que controla uma transformação criptográfica; chaves secretas devem ser imprevisíveis e protegidas.
- **cifra** — Algoritmo criptográfico reversível controlado por chave, usado para transformar texto claro em texto cifrado.
- **cifração** — Operação que transforma texto claro em texto cifrado usando uma chave.
- **codificação** — Representação de dados em outro formato segundo uma regra pública e reversível, sem objetivo de sigilo.
- **confidencialidade** — Propriedade que limita a compreensão dos dados às entidades autorizadas.

- **confusão** — Meta de projeto que dificulta relações simples entre chave, entrada e saída de uma cifra.
- **criptografia** — Área que estuda mecanismos matemáticos para confidencialidade, integridade, autenticidade e outras propriedades de segurança.
- **código-fonte** — Texto escrito numa linguagem de programação e executado ou traduzido para produzir o comportamento de um programa.
- **dados associados** — Dados que permanecem visíveis, mas são autenticados junto do texto cifrado em AEAD.
- **datacenter** — Instalação física que abriga servidores, armazenamento, equipamentos de rede, energia e refrigeração.
- **decifração** — Operação legítima que usa a chave apropriada para recuperar o texto claro.
- **disponibilidade** — Propriedade de manter sistemas e informações acessíveis quando necessários.
- **etiqueta de autenticação** — Valor curto produzido por MAC ou AEAD e verificado antes de aceitar a mensagem; em inglês, *authentication tag*.
- **função em programação** — Bloco de código nomeado que recebe argumentos, executa operações e pode devolver um resultado.
- **função hash** — Função pública que resume entradas de tamanho arbitrário em uma saída fixa e deve resistir a pré-imagens e colisões.
- **hash** — Resumo de tamanho fixo produzido por uma função hash; não é cifra e não possui operação legítima de decifração.
- **HMAC** — *Hash-based Message Authentication Code*: MAC construído de forma padronizada a partir de uma função hash.
- **integridade** — Propriedade que permite detectar modificação não autorizada dos dados.
- **Kerckhoffs** — Princípio segundo o qual a segurança deve depender da chave, não do segredo do algoritmo ou de seu projeto.
- **log** — Registro de eventos produzido por sistema ou aplicação para auditoria e diagnóstico.
- **MAC** — *Message Authentication Code* (código de autenticação de mensagem): etiqueta calculada com chave secreta para integridade e autenticidade.
- **metadados** — Dados sobre a comunicação ou o arquivo, como participantes, horários, tamanhos, tipo e localização.
- **modelo de ameaça** — Descrição dos ativos, adversários, capacidades, objetivos e limites considerados numa análise de segurança.
- **não repúdio** — Evidência que ajuda terceiros a atribuir uma ação, combinando assinatura, identidade, proteção da chave e processo institucional.
- **palavras-chave do Python** — Elementos reservados da linguagem: `def` declara função; `return` devolve resultado; `if/else` decide; `for/while` repete; `pass` marca bloco vazio; `None` representa ausência de valor.
- **protocolo** — Conjunto de regras que define mensagens, estados, verificações e uso combinado de primitivas.

- **Python** — Linguagem de programação usada nas práticas deste material; a versão indicada é Python 3.
- **risco residual** — Risco que permanece depois da aplicação dos controles de segurança.
- **SHA-2** — Segunda família do *Secure Hash Algorithm*, que inclui SHA-256 e SHA-512.
- **SHA-256** — *Secure Hash Algorithm* com resumo de 256 bits, integrante da família SHA-2.
- **SHA-512** — *Secure Hash Algorithm* com resumo de 512 bits, integrante da família SHA-2.
- **teste automatizado** — Verificação executável que compara o comportamento obtido com o comportamento esperado.
- **texto cifrado** — Saída da cifração, normalmente incompreensível sem a chave apropriada.
- **texto claro** — Dados antes da cifração ou depois de uma decifração válida; em inglês, *plaintext*.
- **TODO** — Marcador do inglês *to do* (a fazer), usado no código inicial para indicar um trecho que o estudante deve completar.
- **vulnerabilidade** — Fraqueza que pode ser explorada para violar uma propriedade de segurança.

Objetivos de aprendizagem

- Distinguir confidencialidade, integridade, autenticidade e não repúdio.
- Aplicar o modelo de ameaça para delimitar o que um sistema deve proteger.
- Justificar por que segredo do algoritmo não substitui uma chave secreta.

Pergunta-guia

Que propriedades precisamos preservar quando uma mensagem atravessa um ambiente hostil?

Texto do aluno

Criptografia é o estudo de técnicas matemáticas usadas para proteger informação e apoiar relações de confiança. Ela não é sinônimo de “esconder texto”. Dependendo do problema, queremos impedir leitura, detectar alteração, confirmar quem produziu uma mensagem ou demonstrar que determinada ação foi autorizada. Essas metas são diferentes e exigem mecanismos diferentes.

A **confidencialidade** limita quem compreende os dados. A **integridade** permite detectar modificações não autorizadas. A **autenticidade** dá evidência sobre a origem de uma mensagem. O **não repúdio** busca evidência verificável por terceiros, normalmente com assinaturas digitais e procedimentos institucionais. Disponibilidade também é um objetivo de segurança, mas criptografia sozinha não impede que um atacante desligue um servidor ou sature uma rede.

Antes de escolher um algoritmo, descrevemos o **modelo de ameaça**: quais ativos importam, quem pode atacar, quais capacidades essa pessoa possui e quais riscos ficam fora do escopo. Um invasor pode apenas observar o canal, pode alterar mensagens, pode roubar um dispositivo ou até obter algumas chaves. A mesma solução não atende igualmente a todos esses cenários.

Um princípio clássico, associado a Kerckhoffs, afirma que a segurança deve continuar válida mesmo quando o projeto do sistema é público; somente a chave precisa permanecer secreta. Projetos públicos podem ser avaliados por muitas pessoas. Algoritmos caseiros e “segredos” costumam ocultar falhas, não eliminá-las. Na prática, usamos padrões analisados, implementações mantidas e protocolos que indiquem como chaves, números aleatórios e mensagens se combinam.

Conceitos essenciais

- Texto claro, algoritmo, chave, texto cifrado e decifração.
- Confidencialidade, integridade, autenticidade e não repúdio.
- Atacante passivo (observa) e atacante ativo (altera ou injeta dados).
- Ativo, ameaça, vulnerabilidade, controle e risco residual.
- Princípio de Kerckhoffs: o projeto pode ser público; a chave é o segredo.
- Criptografia é parte de um sistema sociotécnico, não uma solução isolada.

Exemplo guiado

Considere um boletim enviado por rede. Cifrar o arquivo pode impedir sua leitura por terceiros, mas não prova que ele veio da secretaria e não detecta necessariamente uma troca por outro arquivo cifrado. Para o cenário completo podemos precisar de confidencialidade, autenticação da origem, integridade, controle de acesso, registro de operações e cópias de segurança.

Síntese

Comece pelo objetivo e pelo adversário, não pelo nome de um algoritmo. Uma construção é segura apenas em relação a hipóteses explícitas e a um modo correto de uso. Chaves fortes não corrigem senhas expostas, computadores comprometidos ou protocolos mal projetados.

Aprofundamento e aplicações

Segurança como propriedade de um sistema

Uma análise criptográfica começa pelas fronteiras do sistema. O dado pode estar protegido durante a transmissão e permanecer exposto na tela, no arquivo temporário, no log ou na conta de uma pessoa autorizada. Por isso é útil acompanhar o ciclo completo: criação, transmissão, processamento, armazenamento, cópia e descarte. Em cada etapa mudam os adversários e os controles disponíveis. Uma cifra forte no canal não corrige permissões excessivas no banco de dados.

As propriedades também podem entrar em tensão. Um serviço pode registrar todas as ações para aumentar a responsabilização e, ao mesmo tempo, criar um acervo de metadados sensíveis. Pode esconder o conteúdo e ainda revelar quem conversou, quando e por quanto tempo. Projetar segurança exige declarar prioridades, retenção, pessoas autorizadas e risco residual, em vez de usar a palavra “seguro” como se fosse uma propriedade única.

Do requisito ao mecanismo

Um bom requisito é verificável. “Proteger as notas” é vago; “impedir que uma conta de aluno leia notas de outra turma e registrar toda alteração feita por docente” permite pensar em testes. Controle

de acesso trata a autorização, criptografia pode limitar leitura, MAC ou assinatura pode detectar alteração, auditoria registra eventos e backup apoia recuperação. Cada controle possui hipóteses e não substitui os demais.

Também separe acidente de ataque. Uma soma de verificação simples pode detectar erro de transmissão, mas um adversário consegue recalculá-la. Um mecanismo criptográfico inclui segredo ou confiança autenticada para resistir a alteração intencional. Ao avaliar uma solução, pergunte sempre o que uma pessoa sem a chave consegue observar, modificar, repetir ou bloquear.

Aplicações para observar

- Envelope opaco: sugere confidencialidade, mas não garante autoria.
- Lacre numerado: ajuda a detectar violação, mas não esconde o conteúdo.
- Crachá: indica uma identidade alegada; o processo de emissão sustenta a confiança.

Erros e armadilhas a evitar

- Tratar codificação Base64 como criptografia. Ela é reversível sem chave.
- Confundir autenticação de usuário com autenticidade de cada mensagem.
- Afirmar que uma cifra também garante integridade sem examinar o modo de uso.

Roteiro de estudo ativo

Retome a pergunta-guia: *Que propriedades precisamos preservar quando uma mensagem atravessa um ambiente hostil?* Escreva uma resposta inicial sem consultar o material e, em seguida, revise-a usando definições, condições e limites apresentados no encontro. Para cada conceito essencial, registre uma entrada ou situação, a operação realizada, a propriedade obtida e uma hipótese que precisa permanecer verdadeira. Compare pelo menos dois mecanismos próximos e explique por que eles não são intercambiáveis. Por fim, crie um contraexemplo de uso incorreto e descreva como um atacante poderia explorá-lo. O resultado esperado desta revisão é: A turma deve sair capaz de traduzir uma situação informal em propriedades, adversários e controles, sem começar pela escolha de algoritmo.

Lista de atividades

Responda com justificativas. Quando houver cálculo, registre as etapas.

1. Um aplicativo cifra mensagens, mas não autentica o remetente. Quais propriedades ele oferece e quais deixa em aberto?
2. Classifique como ataque passivo ou ativo: (a) copiar tráfego; (b) trocar o valor de uma transferência; (c) medir horários e tamanhos; (d) repetir uma requisição antiga.
3. Construa um modelo de ameaça curto para notas armazenadas em um sistema escolar.
4. Explique o princípio de Kerckhoffs com suas palavras.
5. Por que uma cópia de segurança não substitui integridade criptográfica?
6. Se todos usam a mesma chave, quem consegue provar qual pessoa enviou a mensagem?
7. O que um atacante aprende apenas observando tamanhos e horários?

8. Qual é o ativo mais importante no sistema escolar proposto?
9. Analise a confusão frequente a seguir e escreva uma orientação correta: *Tratar codificação Base64 como criptografia. Ela é reversível sem chave.*
10. Construa um mapa conceitual que relacione os conceitos essenciais deste encontro. Inclua pelo menos seis ligações explicadas.

Atividade 11 — Prática de programação

Objetivo e limite de segurança

Modificar uma transferência, recalcular um hash público e demonstrar por que somente um autenticador com chave impede que o atacante produza uma nova verificação válida. O código deve ser executado somente para aprendizagem. Quando houver construção de brinquedo, ela não oferece segurança real.

Título: Ataque a uma transferência: SHA-256 versus HMAC

Preparação: Python 3, sem pacotes externos.

Tempo sugerido: 30–45 minutos

Antes de programar: bibliotecas, funções e métodos

Leia esta lista antes do código. Ela explica de onde vêm os recursos usados e qual papel cada chamada desempenha no experimento.

- `hashlib` é a biblioteca padrão de hashes; `hashlib.sha256(mensagem)` cria o objeto SHA-256 e `digest()` devolve os 32 bytes do resumo.
- `hmac.new(chave, mensagem, hashlib.sha256)` calcula HMAC-SHA-256. Diferentemente do hash simples, o resultado depende de uma chave secreta.
- `hmac.compare_digest(a, b)` compara etiquetas evitando uma comparação ingênua que possa vaziar informação pelo tempo.
- `secrets.token_bytes(32)` gera 32 bytes imprevisíveis com a fonte segura do sistema operacional. Aqui representa uma chave de 256 bits.

Código inicial

Copie o código, execute-o uma vez e leia a mensagem do primeiro teste que falhar. Substitua somente os trechos marcados com `TODO`.

```
import hashlib
import hmac
import secrets

def sha256(mensagem):
    return hashlib.sha256(mensagem).digest()

def criar_hmac(chave, mensagem):
    # TODO: produza um HMAC-SHA-256.
    pass
```

```
def verificar_hmac(chave, mensagem, etiqueta):
    # TODO: recalcule e use hmac.compare_digest.
    pass

original = b"destino=ana;valor=100"
alterada = b"destino=ana;valor=900"

# SHA-256 nao possui chave. O atacante consegue recalculer o resumo.
hash_original = sha256(original)
hash_forjado = sha256(alterada)
assert hash_forjado == sha256(alterada)

# O servidor e o remetente compartilham esta chave; o atacante nao.
chave_legitima = secrets.token_bytes(32)
chave_atacante = secrets.token_bytes(32)
etiqueta_original = criar_hmac(chave_legitima, original)

assert verificar_hmac(chave_legitima, original, etiqueta_original)
# TODO: crie uma etiqueta para a mensagem alterada com a chave do atacante
# e confirme que o servidor a rejeita.
```

Tarefas

1. Implemente `criar_hmac` e `verificar_hmac` com HMAC-SHA-256 e comparação segura.
2. Assuma que o atacante interceptou a transferência, trocou 100 por 900 e enviou também um SHA-256 recalculado. Mostre por que a dupla mensagem–hash parece válida para quem não possui outra fonte de confiança.
3. Faça o atacante tentar autenticar a mensagem alterada com outra chave e confirme que o servidor rejeita a etiqueta.
4. Preencha uma tabela indicando se SHA-256 e HMAC fornecem confidencialidade, integridade contra atacante, autenticidade e não repúdio.

Testes e evidências esperadas

- A mensagem alterada acompanhada de seu novo SHA-256 deve passar numa verificação que confia apenas no hash recebido.
- O HMAC legítimo deve ser aceito para a mensagem original.
- A mensagem alterada com etiqueta produzida por outra chave deve ser rejeitada.
- O aluno deve concluir que nenhuma das duas alternativas cifra o conteúdo e que HMAC, por usar chave compartilhada, não fornece não repúdio.

Entrega: código executável, saída dos testes e um parágrafo que relacione o resultado à propriedade criptográfica estudada. Não entregue apenas uma captura de tela.

Desafio de fechamento

Escolha uma situação cotidiana (mensageiro, cartão, prontuário ou Wi-Fi) e identifique quatro propriedades de segurança e uma limitação que a criptografia não resolve.

Checklist de aprendizagem

- Consigo distinguir confidencialidade, integridade, autenticidade e não repúdio.
- Consigo aplicar o modelo de ameaça para delimitar o que um sistema deve proteger.
- Consigo justificar por que segredo do algoritmo não substitui uma chave secreta.